SAKARYA ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ
VERİ YAPILARI DERSİ
2. ÖDEV
RAPOR

Öğrenci Adı Soyadı: Zeynep Uludağ

Numarası: B241210029

Grubu: 1-C

Bu projede İkili Arama Ağacı , Kuyruk ve Çift Yönlü Dairesel Bağlı Liste veri yapılarını bir arada kullanarak bir simülasyon geliştirdim. Amaç, dosyadan okunan sayıları belirli kurallara göre organize etmek ve silinen yapıları diğer yapılara dağıtmaktı temelde.

Projeyi daha düzenli yönetebilmek için her veri yapısını ayrı sınıf ve dosyalarda oluşturdum:

Dugum (Dugum.cpp / .hpp): Ağaç yapısının en temel parçasıdır. İçinde bir sayıyı (veri) ve sağ-sol çocuklara giden işaretçileri tutar.

Agac (Agac.cpp / .hpp): İkili Arama Ağacı (BST) mantığını içerir. Dosyadan okunan her satır bir ağacı temsil eder. Sayıları büyüklüklerine göre sağa veya sola ekler. Önemli Fonksiyonlar: PostorderVerileriAl fonksiyonu ile ağaç yıkıldığında içindeki sayıları dışarı aktarırız. Yukseklik ve DugumSayisi fonksiyonları hesaplamalarda kullanılır.

Kuyruk (Kuyruk.cpp / .hpp): İçinde Agac pointerları tutan bir dizidir. Her kuyruk en fazla 6 ağaç alabilir (Dairesel dizi mantığıyla). Ağaçları gruplamak ve "Zirve Değerini" hesaplamaktır. Zirve değeri, kuyruğun başındaki ağacın kökü ile en yüksek ağacın kökü arasındaki ilişkiye göre hesaplanır.

Hucre (Hucre.cpp / .hpp): Ana yönetici listenin düğümleridir.Her hücrenin içinde bir adet Kuyruk bulunur. Çift yönlü olduğu için hem bir önceki hem de bir sonraki hücreyi gösterir.Yonetici (Yonetici.cpp / .hpp): Tüm işlemleri yöneten sınıftır. Dosya okuma, listeyi kurma, ekrana çizdirme ve simülasyonu başlatma görevlerini üstlenir. Hazır string kütüphaneleri yasak olduğu için dosyayı karakter karakter (char) okudum: Dosyadaki rakamları matematiksel işlemle sayıya çevirdim. Her satır bir Ağaç kabul edildi. Satır bitene kadar sayılar o anki ağaca eklendi. Satır bittiğinde (Enter karakteri), bu ağaç aktif Kuyruğa eklendi. Kuyruk dolduğunda (6 ağaç olduğunda), bu kuyruk ana Listeye eklendi ve yeni bir kuyruk oluşturuldu.

Kullanıcıdan tur sayısı alındıktan sonra simülasyon başlar: En Zayıfı Bulma: Listedeki tüm kuyruklar taranır. "Zirve Değeri" en küçük olan kuyruk tespit edilir. Elenen (zayıf) kuyruğun içindeki ağaçlar yıkılır. İçindeki sayılar Postorder sırasıyla alınır. Bu sayılar, elenen kuyruğun bir sonrasındaki kuyruktan başlayarak sırayla diğer kuyruklara tek düğümlü yeni ağaçlar olarak dağıtılır.

Programı oluştururken en çok dikkat ettiğim şey; bellek sızıntısını önlemek için yıkıcı fonksiyonlarda gerekli temizlemeleri yapmaya, her new için bir delete oluşturmaya örneğin, özen gösterdim.